

The exam allows a calculator and two pages of notes (both sides)

Name: J.R. Diaz

1.. Question (20 points): RISC-V Performance Evaluation

A RISC-V processor executes a program with the following instruction mix:

- 40% arithmetic instructions (CPI = 1)
- 25% load/store instructions (CPI = 2)
- 20% branch instructions (CPI = 3)
- 15% jump instructions (CPI = 2)

The processor has a clock rate of **2 GHz**, and the program executes **1×10^6 instructions**.

a. Calculate the average CPI (Cycles Per Instruction) for this program.

$$\text{Avg CPI} = \frac{\text{clock cycles}}{\text{IR count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{IR count}_i}{\text{IR count}} \right)$$

frequency

$$\text{Avg CPI} = (1 \times 0.40) + (2 \times 0.25) + (3 \times 0.20) + (2 \times 0.15)$$

$$0.40 + 0.50 + 0.60 + 0.30$$

$$\boxed{\text{Avg CPI} = 1.80}$$

b. An optimization reduces the CPI of branch instructions from 3 to 2. Recalculate the new average CPI.

$$\text{New Avg CPI} = 0.40 + 0.50 + (2 \times 0.20) + 0.30$$

same same changed same

$$= 0.40 + 0.50 + 0.40 + 0.30$$

$$\text{New Avg CPI} = \boxed{1.60}$$

c. Compute the overall speedup achieved.

$$\text{Overall speedup} = \frac{1.80}{1.60} = \boxed{1.125}$$

2. [20 points] In a **5-stage pipeline with Data Forwarding**, determine how many clock cycles are required to execute the following sequence of instructions:

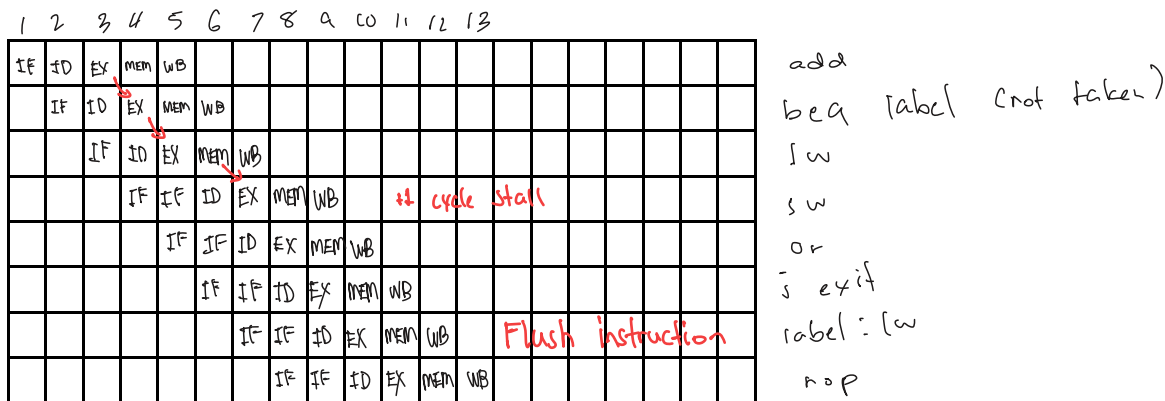
```
add r1,r2,r3
beq r2,r3,label
lw r1,8(r2)
sw r1,12(r4)
or r3,r6,r1
j exit
label: lw r1,8(r12)
```

exit: nop

Assume the following:

- The branch predictor assumes the branch is not taken.
- The branch target address is computed in the EX stage, and the correct PC (if the branch was taken) is loaded in the MEM stage.
- Analyze the scenario where the branch predictor is wrong, and the branch must be taken.

Important: Full credit requires pipeline diagrams showing all forwarding paths, stalls, and flushed instructions.



Total CC: 13

Total number of inst to flush: 1

3. Question (20 points): RISC-V Stack and Function Execution

Conceptual Understanding (5 points)

a. Why does your other memory need a stack? What is the stack used for?

An other memory needs a stack because there are only 32 registers that can hold data. Pushing those values in a stack found in memory frees up those registers which we can then use as local variables when calling functions/subroutines. After the function, we pop those values from the stack (clear space) & store original values to those registers.

b. What is a *local variable*, and why are local variables typically allocated on the stack?

Local variables are variables declared within a function, which when finished, are not saved when resuming to the main program. They are allocated in the stack because pushing and popping stored values in a last in, first out manner allows for clean function calling (nested, recursive).

c. Consider the following RISC-V function. For each line of code, briefly explain its purpose, especially how it interacts with the stack:

Risc V Instruction	Comment
my_function:	label
addi sp, sp, -16	Pushes stack pointer down, allocates space
sw ra, 12(sp)	Pushes return address in stack, storing it in memory
sw s0, 8(sp)	Stores content of register to the stack (above ra)
addi s0, sp, 16	Adds sp & immediate value to s0
li t0, 5	Loads immediate value to temporary register
sw t0, -4(s0)	Stores contents of t0 address of s0 minus plus the offset value
lw ra, 12(sp)	Pops ra value in stack and stores it back to ra
lw s0, 8(sp)	Pops original value of s0 from stack and stores it back to s0
addi sp, sp, 16	Sets stack pointer to top (default address), to clear up space
jr ra	Jumps back to return address (the address after the jump & link instruction)

4. Question (20 points): RISC-V 5-Stage Pipeline (No Forwarding)

Consider a standard 5-stage RISC-V pipeline with the following stages: **IF (Instruction Fetch)**, **ID (Instruction Decode)**, **EX (Execute)**, **MEM (Memory)**, **WB (Write Back)**.

- There is **no forwarding (bypassing)**.
- A register value can only be read **after it has been written back in the WB stage**.
- The processor inserts **stalls (NOPs)** when necessary to resolve hazards.

```
add x5, x1, x2
lw x6, 0(x5)
lw x7, 4(x6)
add x7, x8, x9
```

[illegible]

5.[20 points] Given a simplified pipelined processor datapath, design a forwarding (bypassing) unit that resolves RAW (Read After Write) hazards from the immediately preceding instruction (one instruction above). Ignore load-use (lw) hazards for now. **Indicate where the forwarding unit should be placed, label all its inputs and outputs, specify the internal control logic, and add any required multiplexers to the datapath with proper connections.**

*Did not finish

